



UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERIA MECANICA ELECTRICA, ELECTRONICA Y DE
SISTEMAS
(FIMEES)

Escuela Profesional de Ingenieria de Sistemas

ACTIVIDAD 03 • APRENDIZAJE DE MAQUINA

Algoritmos de enjambre aplicados al aprendizaje automatico

*Feature selection · ajuste de hiperparametros
entrenamiento de redes sin backpropagation · clustering*

Curso: Aprendizaje de Maquina — IX ciclo, grupo B
Docente: Mayenka Fernandez Chambi
Presentado por: Paolo Boris Luna Luque
Facultad: FIMEES
Escuela Profesional: Ingenieria de Sistemas

Repositorio de codigo

<https://github.com/paololuna-luw/algoritmos-enjambre>

incluye README, codigo fuente, figuras y este informe

Puno, Peru — Septiembre de 2026

Contents

1	Introduccion	2
2	Marco breve de los algoritmos	2
2.1	Artificial Bee Colony (ABC)	2
2.2	Particle Swarm Optimization (PSO)	3
3	Feature selection con ABC	3
3.1	Problema	3
3.2	Ciclo del enjambre	3
3.3	Protocolo	3
3.4	Resultados	3
3.5	Fragmento relevante	4
3.6	Discusion	4
4	Hyperparameter tuning con PSO	5
4.1	Problema	5
4.2	Ciclo del enjambre	5
4.3	Resultados	5
4.4	Fragmento relevante	5
4.5	Discusion	6
5	Entrenamiento de una red neuronal sin backpropagation	6
5.1	Problema	6
5.2	Ciclo del enjambre	7
5.3	Resultados	7
5.4	Fragmento relevante	8
5.5	Discusion	8
6	Clustering con PSO	8
6.1	Problema	8
6.2	Ciclo del enjambre	9

6.3	Resultados	10
6.4	Fragmento relevante	10
7	Reproducibilidad y repositorio	11
8	Conclusiones	11

1 Introduccion

Los algoritmos de enjambre resuelven problemas de optimizacion con poblaciones de agentes simples que cooperan. En aprendizaje automatico esa idea aparece de forma natural: elegir variables, ajustar hiperparametros, hallar pesos de una red o colocar centroides son problemas de busqueda.

Este informe desarrolla cuatro ejemplos ejecutables. En cada uno se explicita el ciclo que pide la actividad.

Ciclo del enjambre — presente en los cuatro ejemplos

1. Representacion de la particula (o fuente de alimento).
2. Inicializacion del enjambre.
3. Funcion de aptitud.
4. Comportamiento del agente.
5. Evolucion.
6. Criterio de finalizacion.

Se implementaron **Artificial Bee Colony** (ABC) en representacion binaria para seleccion de características y **Particle Swarm Optimization** (PSO) para las demas tareas, ambos desde cero en Python (NumPy + scikit-learn).

Repositorio GitHub

Codigo, figuras y README:

<https://github.com/paololuna-luw/algoritmos-enjambre>

2 Marco breve de los algoritmos

2.1 Artificial Bee Colony (ABC)

ABC modela tres roles: abejas empleadas, observadoras y exploradoras. Cada fuente de alimento es una solucion candidata. Las empleadas explotan vecindarios; las observadoras eligen fuentes con probabilidad proporcional a la aptitud; las exploradoras abandonan fuentes estancadas y vuelven a muestrear el espacio.

2.2 Particle Swarm Optimization (PSO)

Cada partícula i tiene posición $\mathbf{x}_i$ y velocidad $\mathbf{v}_i$. Recuerda su mejor posición personal $\mathbf{p}_i$ y conoce la mejor global $\mathbf{g}$:

$$\mathbf{v}_i^{t+1} = w \mathbf{v}_i^t + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^t) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^t), \quad (1)$$

$$\mathbf{x}_i^{t+1} = \text{clip}(\mathbf{x}_i^t + \mathbf{v}_i^{t+1}, \mathbf{x}_{\min}, \mathbf{x}_{\max}). \quad (2)$$

w es la inercia, c_1 la memoria cognitiva y c_2 la social.

3 Feature selection con ABC

3.1 Problema

Dado un conjunto de d características, se busca una máscara binaria $\mathbf{m} \in \{0, 1\}^d$ que maximice la exactitud de un clasificador y reduzca la cardinalidad. Problema principal: *Breast Cancer Wisconsin* ($n = 569$, $d = 30$). Corroboración: *Wine* ($n = 178$, $d = 13$).

3.2 Ciclo del enjambre

Ciclo ABC — feature selection

Representación. Fuente de alimento = vector binario de longitud d .

Inicialización. $S = 8$ fuentes aleatorias en $\{0, 1\}^d$, con al menos un bit activo.

Aptitud. Wrapper con regresión logística y CV estratificada de 3 pliegues:

$$f(\mathbf{m}) = \text{Acc}_{CV}(\mathbf{m}) - \lambda \frac{\|\mathbf{m}\|_0}{d}, \quad \lambda = 0.012.$$

Comportamiento. La vecindad voltea un bit. Observadoras con ruleta. Exploradora si no hay mejora en $\text{limit} = 4$ ciclos.

Parada. 8 ciclos. Se conserva la mejor fuente global.

3.3 Protocolo

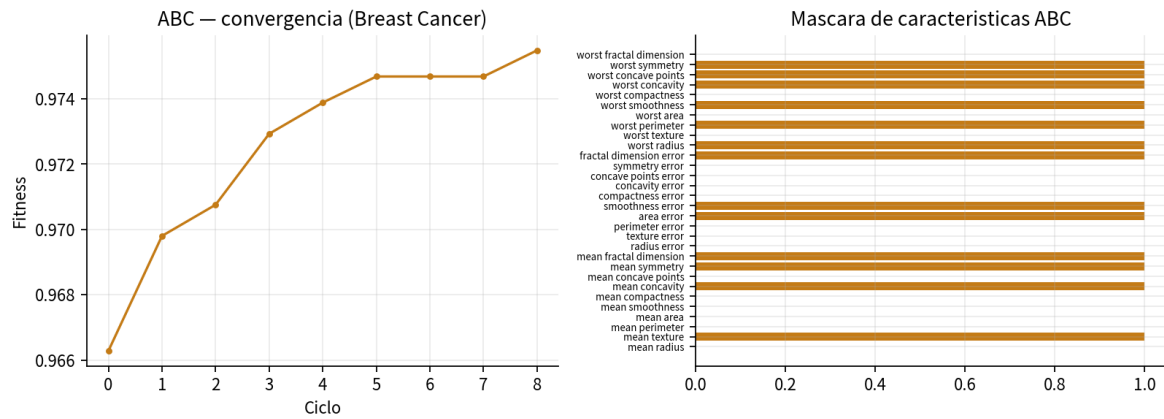
Estandarización z -score. `LogisticRegression(solver=liblinear)`. Semilla 42. Baselines: vector completo y `SelectKBest` con la misma cardinalidad.

3.4 Resultados

ABC redujo Breast Cancer de 30 a 13 variables y subió la exactitud de 0.975 a 0.981. `SelectKBest` con $k = 13$ quedó en 0.949. En Wine pasó de 13 a 6 variables, con 0.983 frente a 0.978 usando todas.

Table 1: Seleccion de caracteristicas con ABC.

Dataset	$ m / d$	Acc. todas	Acc. ABC
Breast Cancer	13 / 30	0.975	0.981
Wine	6 / 13	0.978	0.983

**Figure 1:** Convergencia del fitness ABC y mascara binaria (Breast Cancer).**Variables retenidas — Breast Cancer**

mean texture, mean concavity, mean symmetry, mean fractal dimension, area error, smoothness error, fractal dimension error, worst radius, worst perimeter, worst smoothness, worst concavity, worst concave points, worst symmetry.

3.5 Fragmento relevante

Listing 1: Vecindario binario y fitness wrapper.

```

1  def neighbor(src):
2      k = rng.integers(0, n_bits)
3      nxt = src.copy(); nxt[k] ^= 1
4      if nxt.sum() == 0: nxt[k] = 1
5      return nxt
6
7  def eval_mask(mask):
8      if mask.sum() == 0: return 0.0
9      acc = cross_val_score(clf, X[:, mask.astype(bool)], y, cv=cv).mean()
10     return acc - 0.012 * (mask.sum() / mask.size)

```

3.6 Discusion

El termino de penalizacion evita la solucion trivial de usar todo. Limitacion: cada evaluacion reentrena el wrapper.

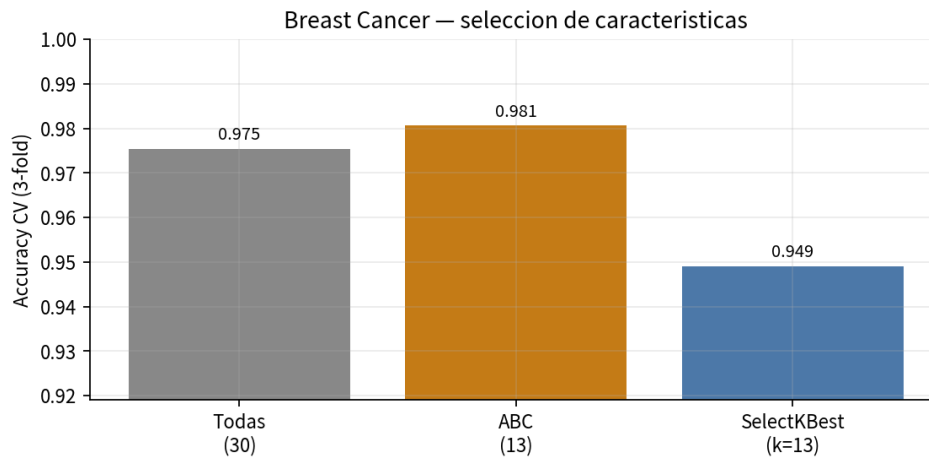


Figure 2: Comparacion de exactitud: todas las variables, ABC y SelectKBest.

4 Hyperparameter tuning con PSO

4.1 Problema

Ajustar C y γ de un SVM RBF sobre Wine, en escala logaritmica.

4.2 Ciclo del enjambre

Ciclo PSO — hiperparametros

Representacion. $\mathbf{x} = (\log_{10} C, \log_{10} \gamma) \in [-2, 3] \times [-4, 1]$.

Inicializacion. 10 partículas uniformes.

Aptitud. $J(C, \gamma) = 1 - \text{Acc}_{CV}(\text{SVM}_{RBF}(C, \gamma))$, 3-fold sobre el 70% de entrenamiento.

Comportamiento. PSO clasico, $w = 0.7$, $c_1 = c_2 = 1.5$.

Parada. 10 iteraciones (110 evaluaciones). Comparacion: Grid 5×5 y SVM por defecto.

4.3 Resultados

PSO encontro $C \approx 92.3$, $\gamma \approx 0.0237$. Exactitud CV 0.992; test 0.981, por encima del grid (0.963).

Table 2: SVM-RBF en Wine. Test = 30% estratificado.

Metodo	Evals.	C, γ	Acc. test
SVM default	1	1, scale	0.981
Grid Search	25	56.2, 0.0316	0.963
PSO	110	92.3, 0.0237	0.981

4.4 Fragmento relevante

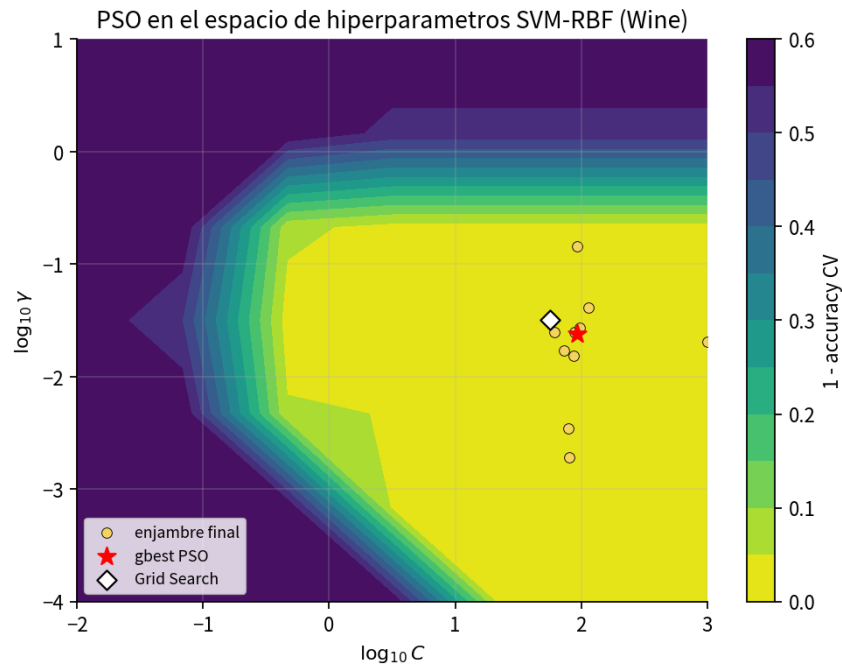


Figure 3: Paisaje de $1 - \text{Acc}_{CV}$. Estrella: gbest PSO. Rombo: grid.

Listing 2: Particula PSO y funcion objetivo SVM.

```

1 def objective(p):
2     C, g = 10**p[0], 10**p[1]
3     acc = cross_val_score(SVC(C=C, gamma=g, kernel="rbf"),
4                           Xtr, ytr, cv=inner).mean()
5     return 1.0 - acc

```

4.5 Discusion

PSO no esta atado a una malla: se mueve en continuo y puede caer entre nodos del grid.

5 Entrenamiento de una red neuronal sin backpropagation

5.1 Problema

Red de **cuatro capas** 4-8-4-3 sobre Iris, sin gradiente. Cada particula es una red completa (91 parametros). Apoyo visual: XOR con MLP 2-4-1.

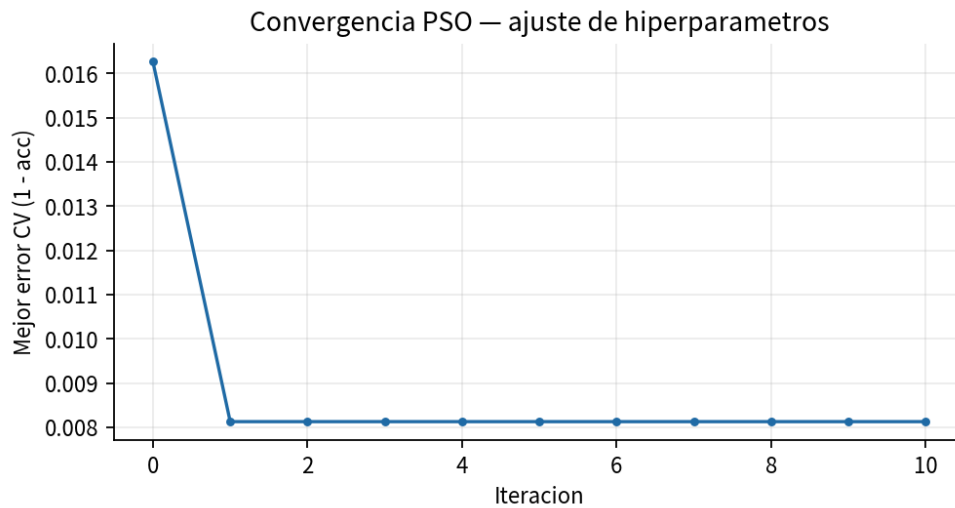


Figure 4: Convergencia del mejor error de validacion cruzada.

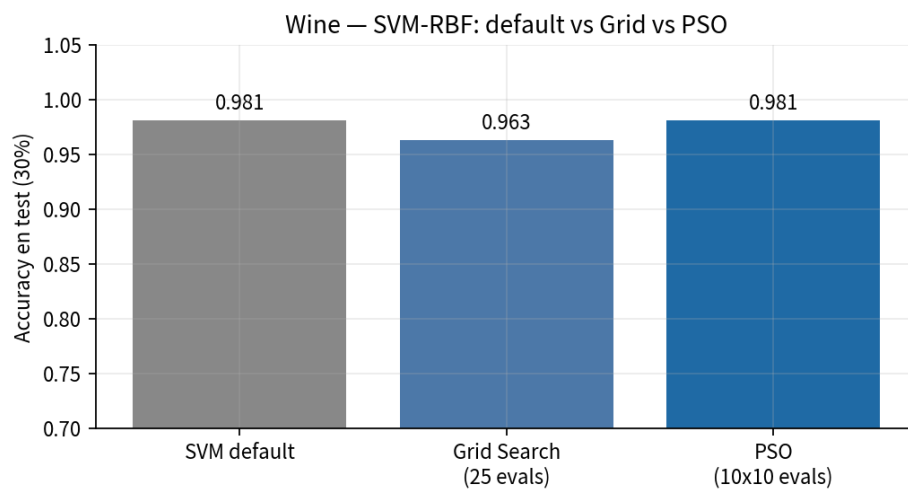


Figure 5: Exactitud de test: default, grid y PSO.

5.2 Ciclo del enjambre

Ciclo PSO — pesos de la red

Representacion. Vector $\mathbf{w} \in \mathbb{R}^{91}$.

Inicializacion. 18 partículas en $[-2.5, 2.5]^{91}$.

Aptitud. Entropia cruzada de entrenamiento. Solo hay *forward*.

Comportamiento. PSO con $w = 0.65$, $c_1 = c_2 = 1.6$.

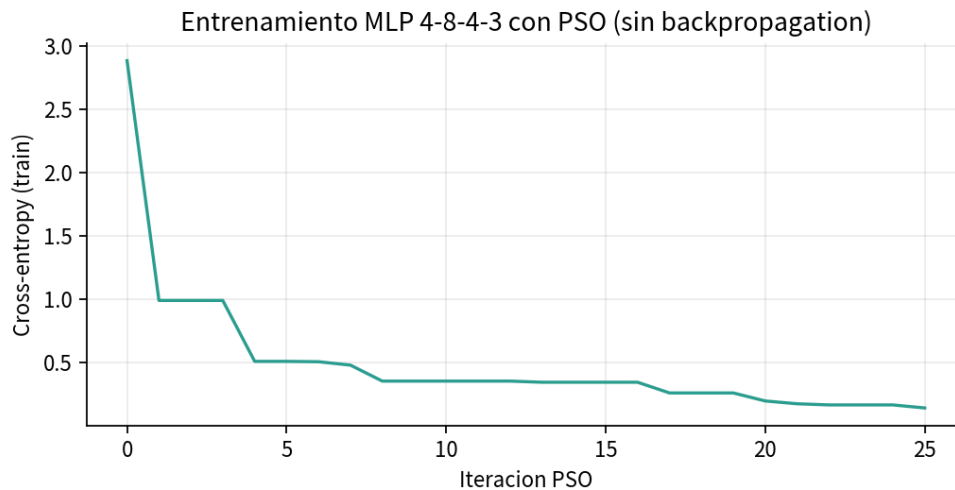
Parada. 25 iteraciones. Comparacion: Adam, 400 epocas.

5.3 Resultados

PSO: CE 0.138, train 0.943, test 0.911. Adam: test 0.756. En XOR el MSE cayo a $8 \cdot 10^{-5}$.

Table 3: MLP 4-8-4-3 en Iris (test 30%).

Optimizador	Parametros	Acc. test
PSO (25 iter, 18 particulas)	91	0.911
Adam / backpropagation (400 epocas)	91	0.756

**Figure 6:** Caída de la entropía cruzada (91 pesos movidos por el enjambre).

5.4 Fragmento relevante

Listing 3: Forward sin backward.

```

1 def objective(w):
2     pred = forward_mlp(Xtr, unpack(w, specs))
3     return -np.mean(np.log(pred[np.arange(len(ytr)), ytr] + 1e-8))

```

5.5 Discusion

PSO no escala a redes enormes. El valor didactico es mostrar que backpropagation es un optimizador, no la definicion de aprender.

6 Clustering con PSO

6.1 Problema

Hallar K centroides que minimicen la inercia. `make_blobs` ($n = 360$, $K = 4$) y Wine en 2 variables ($K = 3$).

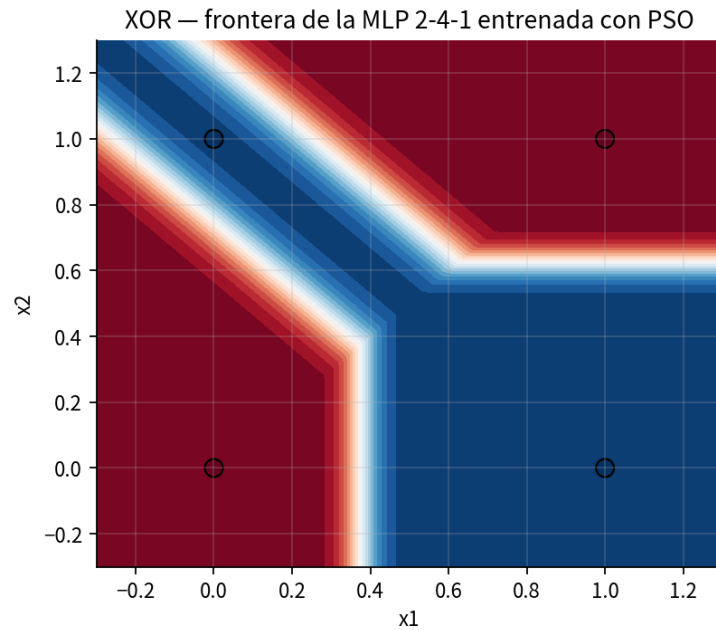


Figure 7: Frontera de la MLP 2-4-1 entrenada con PSO sobre XOR.

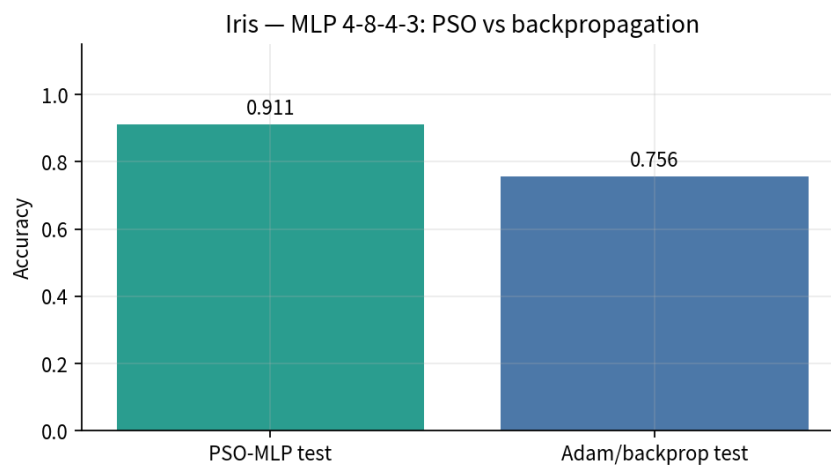


Figure 8: Exactitud de test: PSO versus Adam.

6.2 Ciclo del enjambre

Ciclo PSO — clustering

Representacion. $\mathbf{x} = (\mathbf{c}_1 \| \dots \| \mathbf{c}_K) \in \mathbb{R}^{Kd}$.

Inicializacion. 18 partículas dentro del rango de X .

Aptitud. Suma de distancias al cuadrado al centroide mas cercano.

Comportamiento. Cada partícula mueve todos los centroides.

Parada. 22 iteraciones. Baseline: K-Means `n_init=10`.

6.3 Resultados

En blobs, K-Means gana (inercia 504 vs 2407, silueta 0.824 vs 0.610). En Wine-2D empatan en 0.484. El resultado es util porque no es un empate automatico.

Table 4: Clustering: PSO versus K-Means.

Datos	Inercia PSO	Inercia KM	Sil. PSO	Sil. KM
Blobs $K = 4$	2407	504	0.610	0.824
Wine-2D $K = 3$	—	—	0.484	0.484

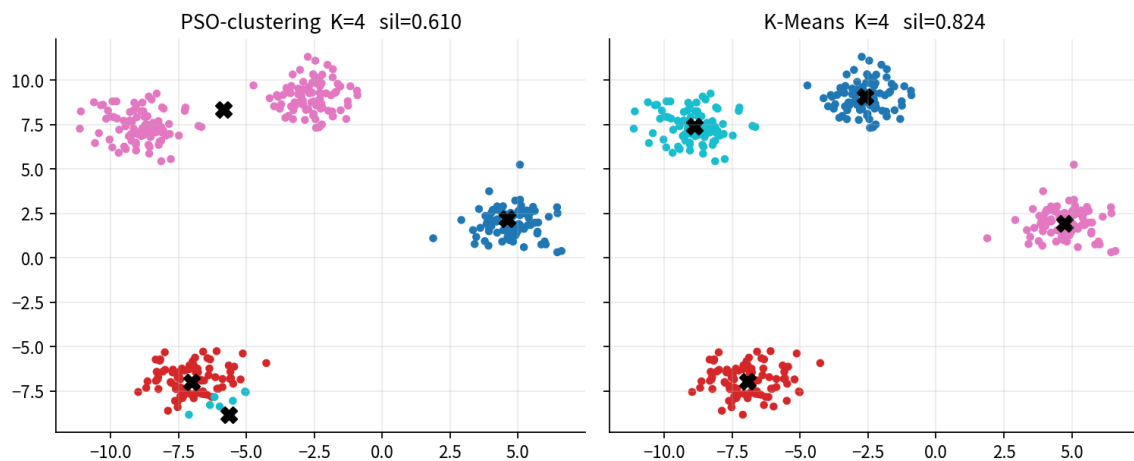


Figure 9: Particiones y centroides de PSO y K-Means sobre blobs.

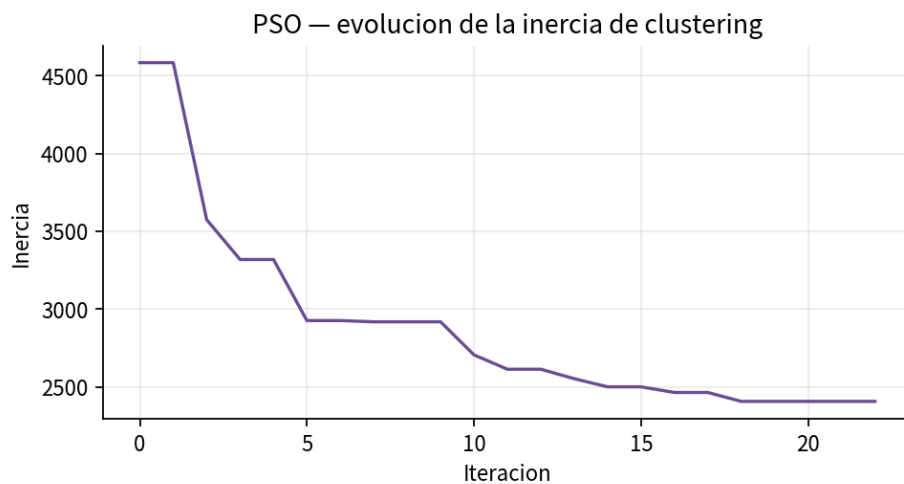


Figure 10: Evolucion de la inercia del mejor individuo.

6.4 Fragmento relevante

Listing 4: Particula = K centroides concatenados.

```

1 def inercia(pos):
2     C = pos.reshape(K, 2)

```

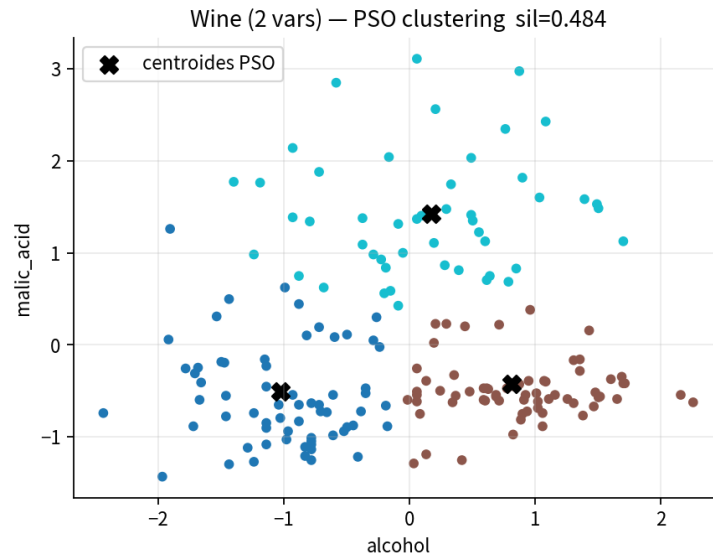


Figure 11: PSO-clustering sobre las dos primeras variables de Wine.

```

3     d = ((X[:, None, :] - C[None, :, :])**2).sum(axis=2)
4     return d.min(axis=1).sum()

```

7 Reproducibilidad y repositorio

Ambiente: Python 3, numpy, scikit-learn, matplotlib. Semilla 42.

```

1 python src/experiments.py

```

Repositorio GitHub

<https://github.com/paololuna-luw/algoritmos-enjambre>

README.md requirements.txt src/experiments.py
 figures/ report/informe.tex report/informe.pdf

8 Conclusiones

1. ABC reduce variables y mejora exactitud frente al vector completo y a SelectKBest.
2. PSO recorre el plano ($\log C, \log \gamma$) y localiza un SVM competitivo.
3. Una MLP de cuatro capas se puede entrenar solo con forward + PSO.
4. El mismo PSO formula clustering como optimizacion de centroides; K-Means sigue siendo mas eficiente aqui.
5. Los cuatro ejemplos comparten el mismo ciclo, que es el objeto de la actividad.

References

- [1] D. Karaboga. *An idea based on honey bee swarm for numerical optimization*. TR06, Erciyes University, 2005.
- [2] J. Kennedy and R. Eberhart. Particle swarm optimization. In *Proc. ICNN*, 1995.
- [3] M. Schiezero and H. Pedrini. Data feature selection based on Artificial Bee Colony algorithm. *EURASIP J. Image Video Process.*, 2013.